

WAIC 2026: Richard Sutton 的 OaK 架构、第一性原理与 Agent 启示

资料范围：71 分钟现场录音、豆包生成的会议 PDF、15 张现场照片，以及 Sutton 的论文和公开演讲。

结论日期：2026-07-20。关键争议以“原始录音/现场照片 > 官方论文与演讲 > 媒体报道 > 自动摘要”为证据优先级。

结论摘要

这场演讲不是在讨论如何扩大 LLM、改进 Transformer 或优化预训练配方，而是在介绍一条以强化学习为基础的通用智能路线：智能体应从第一人称运行时经验中持续学习，形成状态特征、子问题、可复用技能、后果模型和跨时间尺度的规划能力。

OaK (Options and Knowledge) 这个名称和完整组合较新，但组成它的大部分思想并不新：MDP、价值函数、模型式强化学习、Options、General Value Functions (GVFs) 等都有长期研究历史。OaK 的新意主要在于把这些组件组织成开放式、持续运行的认知架构，并强调由智能体自己发现有用抽象。

Claude Code、Cursor 等 coding agent 与 OaK 在任务分解、工具、技能、子代理、记忆和反馈循环上存在功能类比，但没有公开证据表明它们按 OaK 设计。当前 coding agent 通常依赖冻结的预训练 LLM，在上下文和外部存储中适应；OaK 的核心则是运行时持续改变策略、表示、价值和世界模型。

1. 什么是演讲中的“AI 第一性原理”

Sutton 没有给出一个学界公认、唯一且封闭的“AI 公理”。录音原话使用了复数和探索性的表达：simple/basic principles、my quest for those principles。更准确的理解是一条由问题定义、方法原则和架构推论构成的链。

1.1 问题定义：持续交互

智能体通过 observation → action → reward 与世界长期交互。Reward 是定义目标的标量反馈，policy 决定行动，value function 估计未来累计回报。

1.2 世界假设：Big World

真实世界远大于任何有限智能体。策略、价值函数和世界模型只能是近似；即使世界整体稳定，智能体所处的局部环境也可能表现为非平稳。因此，智能体不能只在设计阶段学一次，而必须在运行时持续适应。

1.3 方法原则：The Bitter Lesson

长期、可扩展的突破通常来自能够利用不断增长算力的搜索与学习，而不是把大量人类领域知识固定编码进系统。Bitter Lesson 是“研究资源应投向哪里”的方法论，不是完整的智能理论。

1.4 运行时推论

学习、规划和抽象形成必须能够在 runtime 发生。它们也可以在 design time 发生，但不能只发生在 design time。豆包将这一点概括为“所有能力都只能在部署后形成”，强度过高。

1.5 发展目标

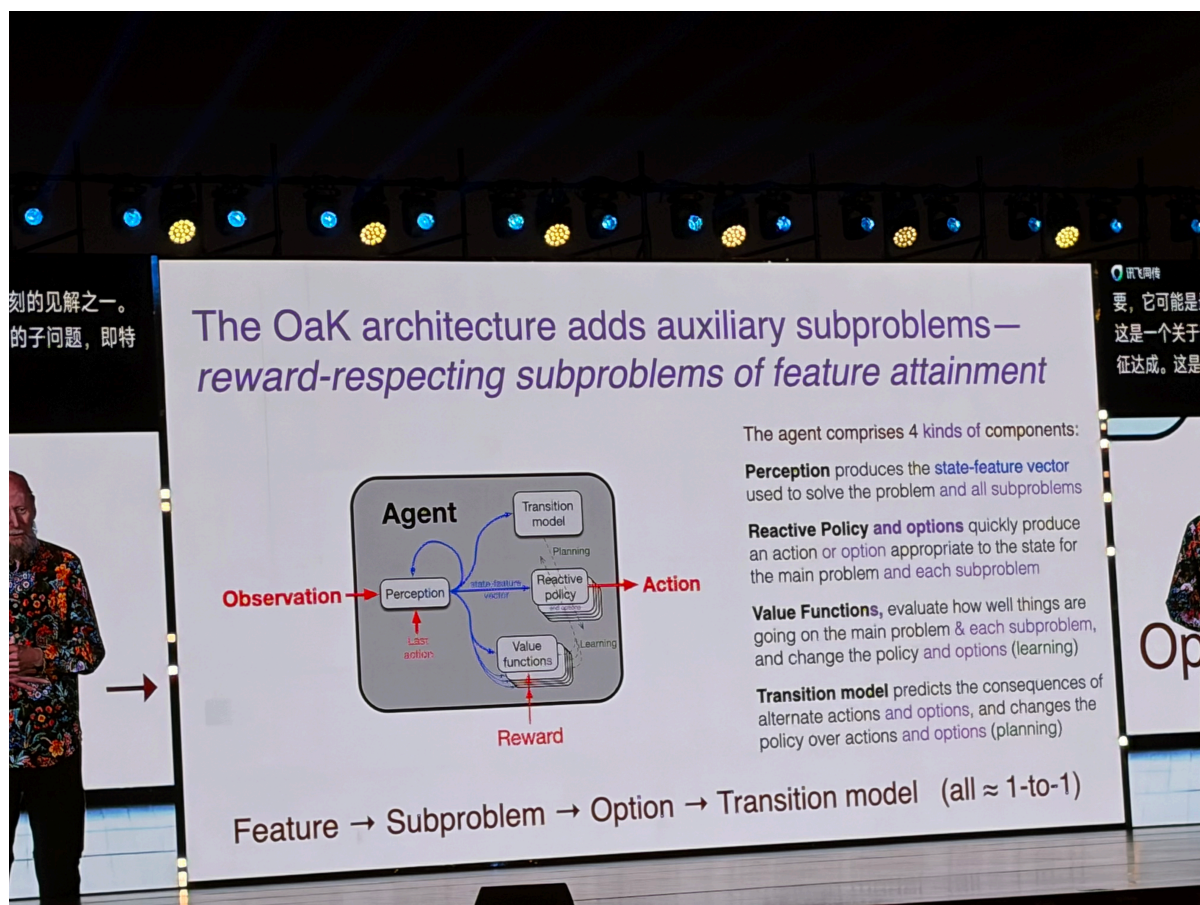
架构应当：

- domain-general：不依赖特定领域手工规则；
- experiential：知识最终扎根于智能体自己的交互经验；
- open-ended：能够持续产生新的状态抽象和时间抽象；
- scalable：搜索和学习能力可随算力增长。

2. OaK 是什么

OaK 可概括为下面的循环：

1. 从经验中形成或发现有用的 **state features**；
2. 为高价值、上下文相关的 feature 建立 **reward-respecting subproblem**；
3. 学习解决子问题的 **option**，即策略和停止条件；
4. 学习 option 的 **transition model**，预测终止状态和累计 reward；
5. 使用 action model 和 option model 做规划；
6. 根据规划和真实执行的效用，保留、合并或淘汰 feature、subproblem、option 和 model。



The Oak architecture involves the following steps all performed in parallel at runtime

- Learn the policy and the value function for maximizing reward
- Generate new state features from existing state features
- Rank-order the features
- Create subproblems, one for each *highly-ranked feature*
- Learn solutions to subproblems (options and sub-value functions)
- Learn transition models of the options and actions
- Plan with the transition models
- Maintain meta-data on the utility of everything; curate the features

这里的 `model` 不是“神经网络模型”的泛称，而是强化学习中的 transition model：预测执行某个 action 或 option 后会到达什么状态，以及途中得到多少累计 reward。

2.1 哪些部分是旧概念

- MDP、policy、value function 和 value iteration;
- model-based reinforcement learning;
- Options framework: 可追溯到 1999 年，使用“策略 + 终止条件”表达时间抽象;
- GVF 与 Horde: 把知识表示为策略相关、可用经验检验的预测;
- continual learning、off-policy learning 和 planning。

2.2 哪些部分是较新的组合

- 把 `feature` → `reward-respecting subproblem` → `option` → `option model` 组织成生成链;
- 让智能体根据主 value function 自动选择值得掌握的子问题;
- 用 utility metadata 持续整理而不是无限积累能力;
- 让新 option 成为更高层规划中的“大步”，再反过来促进更高层抽象。

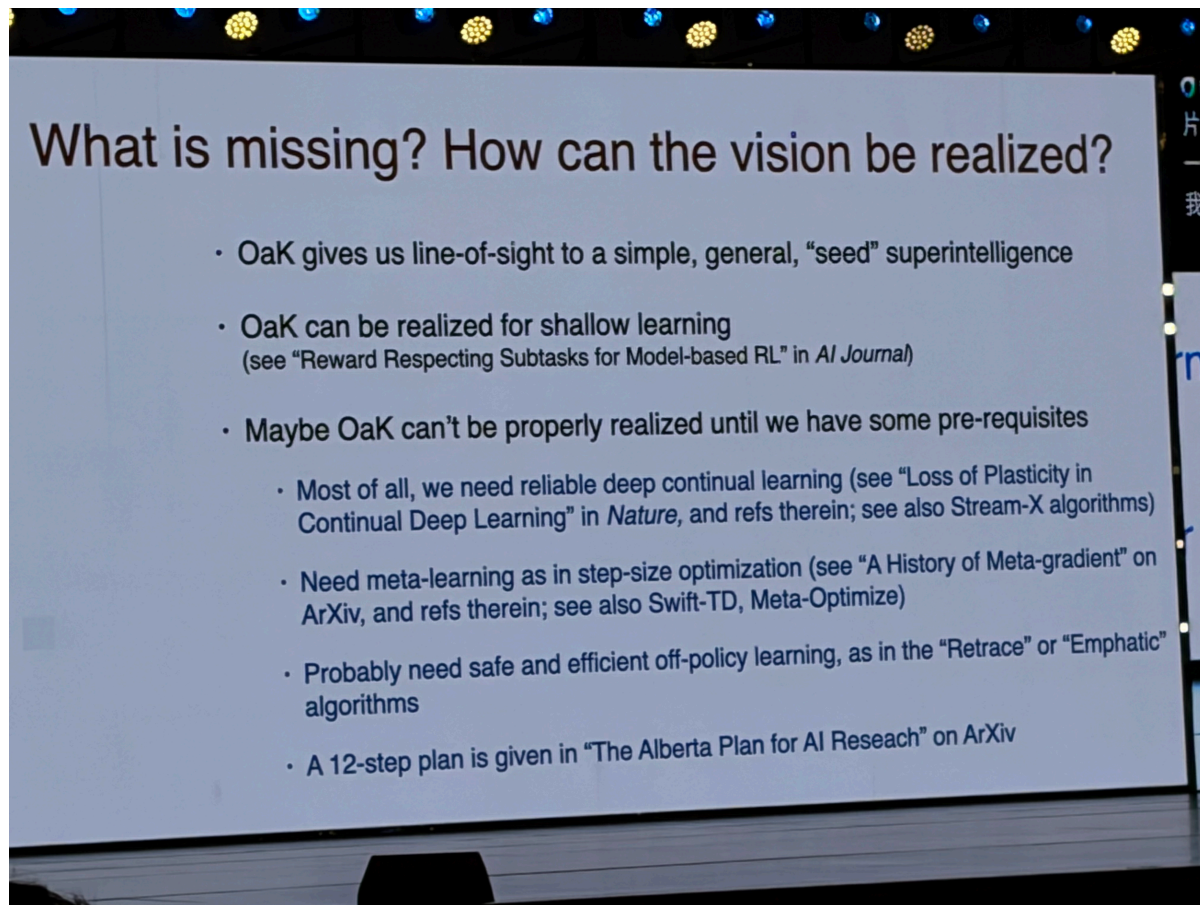
较准确的历史定位是：Options 和其他组件已有几十年历史；Alberta Plan 与 STOMP 路线在 2022 年已经成文；2023 年论文明确讨论 reward-respecting subtasks，并把扩展称为 FC-STOMP/Oak；2025 年 Sutton 以完整 Oak 架构集中公开讲解。

2.3 Oak 仍未解决的问题

Oak 是研究路线，不是可以直接安装的成熟 Agent 框架。关键前置问题包括：

- 深度网络的可靠 continual learning，避免 catastrophic forgetting 和 loss of plasticity;

- 自动发现真正有用的新 state features;
- 大规模、稳定、安全且高效的 off-policy learning;
- 从大量 feature 中选择少数值得建立 subproblem/option/model 的候选;
- 把标量 reward 与复杂真实目标、安全约束和多方价值可靠对齐。



3. 与当前 LLM Agent 的关系

ReAct、Plan-and-Execute、LangGraph、Claude Code 和 Cursor 都实现了某种“观察—推理—行动—再观察”闭环。但 OaK 要求该闭环长期改变智能体本身，而当前 coding agent 多数只在冻结模型之上维护上下文和外部状态。

具体差异如下：

- **Policy**: OaK 的策略从运行时经验持续学习；coding agent 主要由预训练 LLM 在上下文中生成下一步行动。
- **Value function**: OaK 显式学习主问题和子问题的价值；coding agent 可用测试、lint、自评和 verifier 反馈，但通常没有公开的显式 RL value function。
- **Option**: OaK option 是学习得到的策略和停止条件；skill、工具流程、subagent 可作为工程类比，但往往由人预设或由 LLM 临时规划。
- **Transition model**: OaK 学习 action/option 的终止状态分布与累计 reward；coding agent 通常执行工具后读取真实结果，没有公开的在线 option model。
- **Planning**: 两者都有规划，但 OaK 用学习到的世界模型持续规划，LLM agent 主要在 token/context 中推理并调用工具。
- **Runtime learning**: OaK 持续更新权重、表示、技能和模型；coding agent 通常只更新上下文、规则、文件或外部记忆，基础模型权重不变。

- **Subproblem discovery**: OaK 从高价值 feature 生成 reward-respecting 子问题; coding agent 通常根据 prompt 分解任务。

因此, “相似” 只表示功能类比, 不能推出 Claude Code 或 Cursor 是按 OaK 架构实现的。

预训练 LLM 可以作为 OaK 混合系统中的 perception、feature generator 或 policy prior, 但这并不是 Sutton 最纯粹的研究主张。他强调核心知识和能力必须能从第一人称 runtime experience 中成长。

4. 对生产 Agent 最有价值的启示

以下原则现在就能用于 Agent 工程:

1. **奖励一致的子任务**: 子代理不能只优化局部 KPI, 必须继承主目标、硬约束、权限和成本边界。
2. **Option 化技能**: 每个 skill 明确适用条件、输入、步骤、成功标准、失败结果和停止条件。
3. **多时间尺度规划**: 高层使用可复用技能做长程计划; 每个 option 边界重新观察环境, 并允许提前终止和重规划。
4. **建立经验模型**: 记录工具和技能在不同状态下的成功率、耗时、成本、风险及副作用。
5. **效用驱动的调整**: 按对最终结果的实际贡献保留、合并或淘汰 memories、skills 和 subagents。
6. **把硬约束与 reward 分开**: 权限、安全和不可逆操作不能只是一个可被总体得分抵消的软指标。

不应直接照搬 “智能体自己产生目标”。生产系统应由人定义顶层目标、权限和不可违反的约束; 智能体只能在边界内提出子任务。否则开放式目标生成会放大 reward hacking、越权和不可预测行为。

5. 豆包记录的关键核对

明确错误

- “2024 World AI Conference”: 应为 WAIC 2026。2024 是 Sutton 获图灵奖的年份。
- “与 OpenAI AGI roadmap 对齐”: 现场提问对象是 Yann LeCun 的 JEPA 路线。Sutton 回答两者都有 policy、perception、value function 和 transition model 等相似元素, 分歧之一是 LeCun 不喜欢使用 reinforcement learning 这一名称。
- “middle lesson / metal learning / Subhop”: 应分别为 Bitter Lesson、meta-learning、subproblem。

正确或基本正确

- 2030 年 25%、2040 年 50%: 录音清楚说 one chance in four by 2030 和 one chance in two by 2040。部分媒体所写的 10% 与现场录音不符。

表述过强或过度推断

- “所有能力 exclusively 在部署后形成”: 原话是学习、规划、抽象必须能在 runtime 发生, 也可以在 design time 发生。
- “永久 capability library”: OaK 明确维护 utility metadata 并进行 curation, 低价值 feature/option 应被淘汰。
- “自身 intrinsic reward 可消除 reward gaming”: 问答支持减少人类主观打分、让 agent 从世界获得自身可感知的 reward, 但没有证明能消除 reward gaming。
- “经验学习不能给出 TSP 最优解”: 应理解为 OaK 不是精确组合优化算法的通用替代品, 而不是数学上绝不可能得到最优解。

6. 进一步值得追问

如何避免产生无穷无用子目标

Sutton 承认尚无完整答案。现场给出的启发式是优先选择与主 value function 强相关、但权重会随状态变化的 feature；随后还要根据对规划的实际贡献回传效用并淘汰低价值对象。

标量 reward 是否足够表达复杂价值

这是 reward hypothesis，不是已证明定理。多主体价值冲突、安全约束、延迟反馈和 reward hacking 仍是薄弱环节。工程上应将硬约束与优化目标分层。

OaK 为什么与主流 LLM 路线存在张力

LLM 在 design-time 预训练中吸收大量人类知识，部署后通常冻结；OaK 要求核心知识、表示和技能持续从第一人称运行时经验形成。混合路线可以实用，但会偏离其最纯粹的研究假设。

7. 建议学习顺序

1. Sutton 与 Barto, 《Reinforcement Learning》：MDP、value、policy、planning。
2. Options framework：理解“策略 + 停止条件”的时间抽象。
3. GVF 与 Horde：理解如何把知识表示为可验证、策略相关的预测。
4. Reward-respecting Subtasks：理解本场 OaK 最具体、可检验的新机制。
5. Alberta Plan 与 OaK 公开演讲：理解完整研究路线和未解决问题。

主要来源

- Richard Sutton, [The Bitter Lesson](#), 2019.
- Sutton et al., [The Alberta Plan for AI Research](#).
- Sutton et al., [Reward-respecting Subtasks](#).
- Amii, [OaK Architecture — Rich Sutton, RLC 2025](#).
- [RLC 2025 OaK 完整演讲](#).
- [Cursor Agent 官方文档](#).
- [Claude Code subagents 官方文档](#).
- 本地原始资料：现场录音、豆包 PDF、`docs/assets/waic-oak/` 中的 15 张现场照片。

交互版分析保存在 [waic-2026-oak-analysis.canvas.tsx](#)，归档 PDF 为 [waic-2026-oak-analysis.pdf](#)。